

Day 1 & 2 Practice Questions - GATE DA

Day 1 & Day 2 — Topic-Wise Practice Set (GATE DA)

A note on the “PYQ” sets below: I can’t reproduce actual copyrighted exam questions verbatim. So for each topic you get two clearly-labeled sets: **Set A — PYQ-Style**, written to closely match the real exam’s format, phrasing conventions, and difficulty level for that topic (these are practice questions *modeled on* the pattern, not copies of any specific released question), and **Set B — New/Original**, fresh questions extending the same topic with different angles. Both sets are equally good practice; only Set A is deliberately shaped to feel like what you’ll see on the actual paper. Answer keys with one-line reasoning are at the end of each topic.

TOPIC 1 — Python Fundamentals: Data Types, Operators, Control Flow Tracing

(Day 1)

Set A — PYQ-Style (10)

1. What is the output of `print(7 // 2 + 7 % 2)` ?
a. 3.5 (b) 4 (c) 3 (d) 4.5
2. What is the type of the result of `3 / 2` in Python 3?
a. int (b) float (c) complex (d) depends on OS
3. Evaluate: `x = 5; x += 2 ** 2; print(x)` . What is printed?
a. 9 (b) 24 (c) 49 (d) 14
4. What does `not (True and False) or (False and True)` evaluate to?
a. True (b) False (c) None (d) Error
5. Trace the output:

```
x = 10
if x > 5:
    if x > 8:
        print("A")
    else:
        print("B")
else:
    print("C")
```

- a. A (b) B (c) C (d) No output
6. What is `type(5 == 5.0)` in Python?
a. int (b) bool (c) float (d) str
7. What is the output of `print(2 ** 3 ** 2)` ? (Note: `**` is right-associative)
a. 64 (b) 512 (c) 256 (d) 128
8. Given `a = 5; b = 2` , what is `a % -b` ?
a. 1 (b) -1 (c) -1.5 (d) 1.5
9. What does the following loop print?

```
i = 0
while i < 5:
    if i == 3:
        break
    print(i, end=" ")
    i += 1
```

- a. 0 1 2 (b) 0 1 2 3 (c) 0 1 2 3 4 (d) 1 2 3
10. NAT: How many times does “Hi” get printed by `for i in range(2, 10, 3): print("Hi")` ?

Set B — New/Original (10)

11. What is `bool("False")` in Python?
a. True (b) False (c) Error (d) None
12. Trace: `x = 3; y = x; x = x + 1; print(y)` . Output?
a. 3 (b) 4 (c) Error (d) None
13. What does `5 is 5.0` evaluate to?
a. True (b) False (c) Error (d) Depends on interpreter
14. What is the output of `print(10 / 3 == 10 // 3)` ?
a. True (b) False (c) Error (d) 3.33
15. Evaluate `(3 > 2) == (2 > 1)` .
a. True (b) False (c) 1 (d) Error
16. NAT: What is the final value of `x` after `x = 1` then `for i in range(4): x = x * 2` ?
17. What is the output? `python a, b = 5, 10 a, b = b, a print(a, b)`
a. 5 10 (b) 10 5 (c) Error (d) 10 10
18. What does `int("3.5")` do?
a. Returns 3 (b) Returns 3.5 (c) Raises ValueError (d) Returns “3.5”
19. Trace the output of: `python x = 0 for i in range(1, 4): for j in range(1, 3): x += 1 print(x)`
a. 6 (b) 4 (c) 7 (d) 12

20. What is `-7 // 2` in Python (note floor division rounds toward negative infinity)?

- a. -3 (b) -4 (c) -3.5 (d) 3

Answer Key — Topic 1

1(b) `//` → 3, `%` → 1, `sum=4` · 2(b) true division always returns float · 3(a) `5+4=9` · 4(b) `F` or `F = False` · 5(a) nested if, `x=10>8` · 6(b) equality returns bool · 7(b) right-associative: `3*2=9`, `2*9=512` · 8(a) Python `%` sign follows divisor: `-1%... → a%-b` where `5%-2=-1`... recompute: `5 % -2 = -1` in Python; check option — correct answer is -1, i.e. (b) · 9(a) breaks before printing `3 · 10`. Answer: 3 (i.e. 2,5,8) · 11(a) any non-empty string is truthy · 12(a) `y` bound to old value `3` · 13(b) different types, `is` checks identity not equality, and int/float are different objects · 14(b) `3.33≠3` · 15(a) `True==True` · 16. Answer: 16 (`1→2→4→8→16`) · 17(b) swapped via tuple unpacking · 18(c) `int()` can't parse decimal string directly · 19(a) `3×2=6` · 20(b) floors toward `-∞`: `-3.5 → -4`

TOPIC 2 — Arrays/Lists: Indexing, Slicing, Two-Pointer Intro

(Day 1)

Set A — PYQ-Style (10)

- Given `L = [10, 20, 30, 40, 50]`, what is `L[-2]` ?
a. 40 (b) 50 (c) 30 (d) Error
- What is `L[1:4]` for `L = [1,2,3,4,5]` ?
a. `[1,2,3,4]` (b) `[2,3,4]` (c) `[2,3,4,5]` (d) `[1,2,3]`
- What is the output of `L[: -1]` for `L = [1,2,3]` ?
a. `[1,2,3]` (b) `[3,2,1]` (c) Error (d) `[3,1,2]`
- What is the time complexity of accessing `L[i]` in a Python list (array-backed)?
a. $O(n)$ (b) $O(\log n)$ (c) $O(1)$ (d) $O(n \log n)$
- Two-pointer approach to check if a sorted array has a pair summing to target `k` runs in:
a. $O(n^2)$ (b) $O(n \log n)$ (c) $O(n)$ (d) $O(1)$
- What does `L[2:2]` return for any list `L` with `len ≥ 2`?
a. Error (b) `[]` (c) `[L[2]]` (d) None
- Given `L = [5, 3, 8, 1, 9]`, what does `L[1:] = [0]` change `L` to?
a. `[5, 0]` (b) `[5, 0, 3, 8, 1, 9]` (c) `[0, 3, 8, 1, 9]` (d) Error
- What is the space complexity of the two-pointer technique on an array of size `n` (excluding the array itself)?
a. $O(n)$ (b) $O(1)$ (c) $O(\log n)$ (d) $O(n^2)$
- What is `len(L[:2])` for `L` of length 7?
a. 3 (b) 4 (c) 7 (d) 2
- NAT: For `L = [1,2,3,4,5,6]`, using two pointers starting at index 0 and 5 to reverse in place, how many swaps occur?

Set B — New/Original (10)

- What does `L.pop()` cost in time complexity for a Python list of size `n`?
a. $O(1)$ (b) $O(n)$ (c) $O(\log n)$ (d) $O(n^2)$
- Given `L = [1,2,3,4,5]`, what is `L[10:20]` ?
a. `IndexError` (b) `[]` (c) None (d) `[1,2,3,4,5]`
- What is the output of `[1,2,3] + [4,5]` ?
a. `[5,7,3]` (b) `[1,2,3,4,5]` (c) Error (d) `[1,2,3,[4,5]]`
- Using two pointers to detect if an array is a palindrome, what's the worst-case number of comparisons for size `n`?
a. `n` (b) `n/2` (c) `n2` (d) `log n`
- What does `L[:] = [9,9]` do to a list `L` referenced elsewhere by another variable `M = L` ?
a. Only `L` changes (b) Both `L` and `M` reflect new content (c) Error (d) `M` becomes empty
- What is `L[3:1]` for `L=[1,2,3,4,5]` ?
a. `[4,3]` (b) `[]` (c) Error (d) `[3,4]`
- Removing the middle element from a Python list of size `n` takes:
a. $O(1)$ (b) $O(n)$ (c) $O(\log n)$ (d) $O(n^2)$
- NAT: In two-pointer sum-to-target search on a sorted array of 8 elements, what is the maximum number of pointer moves before pointers cross (worst case, target not found)?
- What's the output of `list(range(10))[2:8:2]` ?
a. `[2,4,6]` (b) `[2,4,6,8]` (c) `[3,5,7]` (d) `[2,3,4,5,6,7]`
- Given a sorted array, why does the two-pointer technique work for the "pair sum" problem instead of needing nested loops?
a. Because sorted order lets you eliminate half the search space each step like binary search
b. Because sorted order guarantees moving left pointer right increases sum and moving right pointer left decreases sum, monotonically narrowing the search
c. Because arrays are always small
d. It doesn't actually work without hashing too

Answer Key — Topic 2

1(a) index -2 = second-last · 2(b) slice excludes end index · 3(b) full reverse · 4(c) direct offset computation · 5(c) linear scan with two pointers · 6(b) empty slice · 7(c) slice-assignment replaces from index 1 onward · 8(b) constant extra space · 9(b) indices `0,2,4,6 → 4` elements · 10. Answer: 3 (pairs: `(0,5),(1,4),(2,3)`) · 11(b) shifts all remaining elements · 12(b) out-of-range slice returns empty, no error · 13(b) list concatenation · 14(b) pointers meet at middle · 15(b) slice-assignment mutates in place, both names see it · 16(b) `start>stop` with default step → empty · 17(b) requires shifting elements · 18. Answer: 7 (`n-1` moves max before crossing) · 19(a) `start2 step2` up to `<8` · 20(b) monotonic narrowing is the correct mechanism

TOPIC 3 — Intelligent Agents, Agent Types, PEAS Framework

(Day 1)

Set A — PYQ-Style (10)

- In the PEAS framework, "P" stands for:
a. Process (b) Performance measure (c) Precision (d) Planning
- A thermostat that only reacts to current temperature with no memory of past states is best classified as:
a. Goal-based agent (b) Simple reflex agent (c) Utility-based agent (d) Learning agent
- An agent that maintains an internal state to track aspects of the world not visible in the current percept is a:
a. Simple reflex agent (b) Model-based reflex agent (c) Goal-based agent (d) Utility-based agent
- Which agent type is needed when multiple action sequences could achieve the goal, and you must pick the "best" one?

- a. Simple reflex (b) Model-based reflex (c) Goal-based (d) Utility-based
- 5. For a self-driving car, which is correctly classified as an “actuator” in PEAS?
 - a. Camera (b) Steering wheel (c) Passenger comfort (d) Road conditions
- 6. An environment where the next state depends only on the current state and the agent’s action (not history) is:
 - a. Episodic (b) Markovian (c) Static (d) Deterministic
- 7. A fully observable environment means:
 - a. The agent can see the entire state space instantly at every step
 - b. The agent’s sensors give it access to the complete state of the environment relevant to its action choice
 - c. The environment never changes
 - d. There is only one agent
- 8. Which environment property applies to chess (two-player, no hidden information)?
 - a. Stochastic, fully observable (b) Deterministic, partially observable (c) Deterministic, fully observable (d) Stochastic, partially observable
- 9. A learning agent’s “critic” component is responsible for:
 - a. Choosing actions (b) Providing feedback on how well the agent is doing relative to a fixed performance standard (c) Modeling the environment (d) Generating percepts
- 10. NAT/MCQ: How many main components does a standard “learning agent” architecture have (performance element, learning element, critic, problem generator)?
 - a. 2 (b) 3 (c) 4 (d) 5

Set B — New/Original (10)

- 11. For an automated vacuum cleaner, which of the following is a “sensor” in PEAS terms?
 - a. Wheels (b) Dirt detection sensor (c) Cleanliness of room (d) Suction motor
- 12. Which environment type describes a taxi-driving domain where other cars’ behavior isn’t fully predictable?
 - a. Deterministic (b) Stochastic (c) Static (d) Discrete
- 13. A goal-based agent differs from a utility-based agent primarily because:
 - a. Goal-based agents only care about reaching a goal state, without comparing how “good” different successful paths are, while utility-based agents rank outcomes by preference
 - b. Utility-based agents don’t use goals at all
 - c. Goal-based agents are always faster
 - d. There is no real difference
- 14. An environment in which an agent’s actions do not change future percepts it hasn’t already caused (episodic) means:
 - a. Each episode is independent of previous episodes’ actions
 - b. The environment is static
 - c. The environment is discrete
 - d. The agent has no sensors
- 15. Which of these is a “single-agent” vs “multi-agent” environment distinction example? Chess is multi-agent because:
 - a. It has two players whose actions affect each other’s outcomes
 - b. It has 64 squares
 - c. It’s deterministic
 - d. It’s turn-based only
- 16. In PEAS for a medical diagnosis system, “Environment” refers to:
 - a. The doctor (b) Patients, hospital records, staff, equipment (c) The performance measure (d) The diagnosis output
- 17. A “static” environment is one where:
 - a. The environment doesn’t change while the agent is deliberating
 - b. The environment has no agents
 - c. There’s only one possible action
 - d. The agent never learns
- 18. Which agent architecture explicitly reasons about “what will happen if I do X” using a model of how the world evolves?
 - a. Simple reflex (b) Model-based reflex (with condition-action rules only) (c) Goal-based (d) None of these
- 19. A “discrete” environment (as opposed to continuous) example is:
 - a. Robot arm angle control (b) Chess board positions (c) Autonomous drone flight (d) Temperature control system
- 20. True or False: An environment can be both partially observable and stochastic at the same time (e.g., poker).
 - a. True (b) False

Answer Key — Topic 3

1(b) · 2(b) no internal state, pure condition-action · 3(b) internal state tracks unseen aspects · 4(c) goal-based deliberates over sequences · 5(b) steering wheel acts on world · 6(b) Markov property definition · 7(b) precise definition, not (a)’s overstatement · 8(c) chess is deterministic and fully observable · 9(b) critic gives feedback vs performance standard · 10(c) four components · 11(b) sensor detects dirt · 12(b) unpredictable other agents → stochastic · 13(a) correct distinction · 14(a) episodic = independent episodes · 15(a) multi-agent because of interacting objectives · 16(b) environment = everything agent operates in/on · 17(a) definition of static · 18(c) goal-based agents plan over consequences · 19(b) finite discrete positions · 20(a) True — poker is both

TOPIC 4 — Functions, Recursion Basics, Scope Rules

(Day 2)

Set A — PYQ-Style (10)

1. What is the output?

```
def f(x):
    x = x + 1
    return x
a = 5
f(a)
print(a)
```

- a. 5 (b) 6 (c) Error (d) None

2. What does the following recursive function return for `f(4)` ?

```
def f(n):
    if n == 0:
        return 1
    return n * f(n-1)
```

- a. 4 (b) 24 (c) 10 (d) 16

3. What happens if a recursive function has no base case?

- a. It runs once (b) Infinite recursion / stack overflow (c) Returns None (d) Compile error

4. In Python, a variable assigned inside a function without the `global` keyword is:

- a. Global by default (b) Local to that function (c) Shared across all functions (d) A syntax error

5. What is the output?

```
x = 10
def f():
    print(x)
f()
```

- a. 10 (b) Error (c) None (d) 0

6. What is the output?

```
x = 10
def f():
    x = x + 1
    print(x)
f()
```

- a. 11 (b) 10 (c) UnboundLocalError (d) None

7. How many recursive calls (including the initial call) does `fib(4)` make using naive recursion, where `fib(n) = fib(n-1) + fib(n-2)`, `fib(0)=0`, `fib(1)=1`?

- a. 5 (b) 9 (c) 7 (d) 15

8. What does a default mutable argument (e.g., `def f(x=[])`) risk in Python?

- a. Nothing, it's re-created each call (b) The same list object persists across calls, causing unexpected accumulation (c) Syntax error (d) It becomes immutable

9. What is the time complexity of computing `factorial(n)` recursively (ignoring multiplication cost)?

- a. $O(1)$ (b) $O(\log n)$ (c) $O(n)$ (d) $O(n^2)$

10. NAT: What is the maximum recursion depth reached (number of frames on the stack at the deepest point) for `f(5)` where `f(n)` calls `f(n-1)` until `n==0`?

Set B — New/Original (10)

11. What is the output? `python` `def outer():` `x = 5` `def inner():` `nonlocal x` `x += 1` `return x`
`return inner()` `print(outer())`
a. 5 (b) 6 (c) Error (d) None
12. What is the return value of `g(3)`? `python` `def g(n):` `if n <= 1:` `return n` `return g(n-1) + g(n-2)`
a. 2 (b) 3 (c) 1 (d) 4
13. Which keyword lets an inner function modify a variable in its *enclosing* (not global) scope?
a. global (b) nonlocal (c) static (d) outer
14. What's the output? `python` `def f(a, b=10):` `return a + b` `print(f(5))`
a. 5 (b) 10 (c) 15 (d) Error
15. What does this print? `python` `def f():` `global x` `x = 100` `x = 1` `f()` `print(x)`
a. 1 (b) 100 (c) Error (d) None
16. What is the space complexity of a recursive function with recursion depth n and $O(1)$ work per call (excluding result storage)?
a. $O(1)$ (b) $O(n)$ (c) $O(\log n)$ (d) $O(n^2)$
17. What happens when calling `f(3)`? `python` `def f(n):` `print(n)` `if n > 0:` `f(n-1)` Output order?
a. 0 1 2 3 (b) 3 2 1 0 (c) 3 2 1 (d) 0 1 2
18. In Python, can a function be defined inside another function and returned as a value (closure)?
a. Yes (b) No (c) Only in Python 2 (d) Only for lambda
19. What is the output? `python` `def f(x, y=[]):` `y.append(x)` `return y` `print(f(1))` `print(f(2))`
a. [1] then [2] (b) [1] then [1, 2] (c) Error (d) [1,2] then [1,2]
20. NAT: How many total function calls occur (including the initial call) when computing `f(3)` for `f(n) = f(n-1) + f(n-1)` with base case `f(0) = 1`? (i.e., a binary-branching recursion of depth 3)

Answer Key — Topic 4

1(a) Python passes object references but reassignment inside doesn't affect caller's binding for immutables · 2(b) $4 \times 3 \times 2 \times 1 = 24$ · 3(b) stack overflow · 4(b) local scope by default · 5(a) reading a global from inner scope is fine · 6(c) assignment anywhere in function body makes it local, so reading before assignment fails · 7(c) `fib(4)` naive call tree has 9 calls total — recheck: standard count for `fib(4)` is 9 calls; if options don't include 9 pick closest — here (b)=9 is correct · 8(b) classic Python gotcha · 9(c) linear chain of calls · 10. Answer: 6 ($n=5, 4, 3, 2, 1, 0$) · 11(b) nonlocal lets inner modify enclosing $x \rightarrow 6$ · 12(a) $g(3)=g(2)+g(1)=(g(1)+g(0))+1=(1+0)+1=2$ · 13(b) nonlocal keyword · 14(c) $5+10$ · 15(b) global reassignment persists · 16(b) one frame per call · 17(b) prints before recursing: 3 2 1 0 · 18(a) yes, closures are standard Python · 19(b) mutable default persists across calls → classic gotcha · 20. Answer: $15 (2^4 - 1)$ total calls for depth-3 binary tree starting count at `f(3)`

TOPIC 5 — Strings: Manipulation + Two-Pointer

(Day 2)

Set A — PYQ-Style (10)

1. What is `"hello"[1:4]`?
a. "ell" (b) "hell" (c) "ello" (d) "ell o"
2. What is the output of `"abc" + "def"`?
a. "abcdef" (b) Error (c) "abc def" (d) "defabc"
3. What is `"Hello".lower()`?
a. "HELLO" (b) "hello" (c) "Hello" (d) Error
4. Two-pointer check for whether a string is a palindrome runs in:
a. $O(n^2)$ (b) $O(n)$ (c) $O(\log n)$ (d) $O(1)$
5. What is `len("GATE DA")`?
a. 6 (b) 7 (c) 8 (d) 5
6. What does `"abcabc".count("a")` return?
a. 1 (b) 2 (c) 3 (d) 0
7. Is a Python string mutable?
a. Yes (b) No (c) Only with `.replace()` (d) Only for single characters
8. What does `"racecar"[:-1] == "racecar"` evaluate to?
a. True (b) False (c) Error (d) None
9. What is `" gate ".strip()`?
a. "gate" (b) " gate" (c) "gate" (d) Error
10. NAT: Using two pointers from both ends of a 9-character string to check palindrome property, what is the maximum number of character comparisons needed?

Set B — New/Original (10)

11. What does `"Data".replace("a", "X")` return?
a. "DXtX" (b) "DataX" (c) "Dta" (d) "DXta"
12. What is the output of `"1,2,3".split(",")`?
a. "1,2,3" (b) ['1','2','3'] (c) [1,2,3] (d) Error
13. Which two-pointer technique correctly reverses a string in place *if strings were mutable* (conceptually, using a char list)?
a. Swap characters at positions i and $n-1-i$ while $i < n-1-i$
b. Swap all characters with position 0
c. Only move the left pointer
d. Sort the characters
14. What does `"abc" * 3` produce?

- a. "abcbabc" (b) Error (c) "abc3" (d) 3
15. What is `"Hello World".split()` (no argument)?
- a. ['Hello', 'World'] (b) ['Hello World'] (c) Error (d) ['H','e',...]
16. Checking if two strings are anagrams using character counts (hashing) takes:
- a. $O(n^2)$ (b) $O(n)$ (c) $O(1)$ (d) $O(n \log n)$ — note: sorting-based approach is $O(n \log n)$; hashing-based is $O(n)$; this Q asks about the hashing approach
17. What is `"GATE"[:2]` ?
- a. "GT" (b) "AE" (c) "GA" (d) "GATE"
18. NAT: How many two-pointer swaps are needed to reverse a string of length 6?
19. What does `"gate" in "gate da prep"` evaluate to?
- a. True (b) False (c) Error (d) None
20. Given two pointers scanning a string to find the longest substring without repeating characters (sliding window variant), what auxiliary data structure is typically used to track seen characters?
- a. Stack (b) Set/hash map of last-seen index (c) Sorted array (d) Linked list

Answer Key — Topic 5

1(a) indices 1,2,3 · 2(a) concatenation · 3(b) lowercases all · 4(b) linear scan · 5(b) "GATE DA" has 7 chars including space · 6(b) two 'a's · 7(b) strings are immutable in Python · 8(a) racecar is a palindrome · 9(a) strips leading/trailing whitespace · 10. Answer: 4 (9-char string: compare pairs (0,8),(1,7),(2,6),(3,5); middle index 4 unpaired) · 11(a) all 'a's replaced · 12(b) list of string pieces · 13(a) standard two-pointer swap · 14(a) repetition · 15(a) splits on whitespace by default · 16(b) hashing/counting approach is linear · 17(c) indices 0,2 → "GA" · 18. Answer: 3 (positions (0,5),(1,4),(2,3)) · 19(a) substring present · 20(b) sliding window with hash map is the standard pattern

TOPIC 6 — Problem Formulation & State-Space Representation

(Day 2)

Set A — PYQ-Style (10)

- A well-defined problem formulation consists of:
 - Initial state, actions, transition model, goal test, path cost
 - Only initial state and goal state
 - Only the search algorithm used
 - Only the heuristic function
- In the 8-puzzle problem, a "state" represents:
 - A single tile's position
 - The complete arrangement of all tiles on the board
 - The goal configuration only
 - The move sequence so far
- The "branching factor" of a search problem refers to:
 - The number of goal states
 - The average number of successors per state
 - The depth of the search tree
 - The number of actions in total across the whole space
- In state-space search, the "path cost" function typically:
 - Assigns a numeric cost to a path, often as a sum of step costs
 - Is always 1 for every problem
 - Is irrelevant to search algorithms
 - Only applies to informed search
- For the "missionaries and cannibals" problem, which is part of the correct problem formulation?
 - States = all combinations of missionaries/cannibals/boat positions on each bank
 - States = just the boat's position
 - There is no goal test needed
 - Actions don't need constraints
- A "successor function" (or transition model) does what?
 - Returns the goal state directly
 - Given a state and action, returns the resulting state(s)
 - Computes the heuristic value
 - Counts the total number of states
- The state space of a problem is best described as:
 - The set of all states reachable from the initial state via any sequence of actions
 - Only the initial and goal states
 - The search algorithm's memory usage
 - The heuristic function's range
- Which of these problems has an infinite state space (in its natural formulation)?
 - 8-puzzle
 - Tic-tac-toe
 - Real-valued robot navigation with continuous coordinates
 - Rubik's cube
- A "goal test" can be:
 - An explicit set of goal states, or a property the state must satisfy
 - Always identical to the initial state
 - Irrelevant in state-space search
 - Only used in adversarial search
- MCQ: For a problem where each state has exactly 3 possible actions and the search tree has depth 4, what is the maximum number of leaf nodes (branching factor $b=3$, depth $d=4$)? [b^d]
 - 12
 - 81
 - 64
 - 243

Set B — New/Original (10)

- Which of the following is an example of an "action" in the vacuum-world problem formulation (2-square version)?
 - Suck, Left, Right
 - Clean, Dirty
 - State A, State B
 - Goal reached
- In problem formulation, "abstraction" refers to:
 - Removing irrelevant detail from the real world to define states/actions at the right level
 - Making the problem harder
 - Adding more states
 - Ignoring the goal test
- For the 8-queens problem in its "incremental formulation," a state typically represents:
 - A complete random board
 - A partial arrangement of queens placed so far, one per column up to the current column
 - Only the final solution
 - The number of queens left
- Which pair correctly matches problem type to state-space size behavior?
 - Larger board sizes in N-queens increase the state space combinatorially
 - State space size is always constant regardless of problem size
 - Continuous-valued problems always have smaller state spaces than discrete ones
 - Branching factor has no effect on tree size
- In route-finding problems (e.g., driving from city A to city B), the path cost function usually represents:
 - Number of cities visited only
 - Distance, time, or toll cost accumulated along the route
 - Random values
 - Always equals the number of actions taken
- What does it mean for a problem to have a "single-state" (fully observable, deterministic) formulation?
 - The agent always knows exactly which state it's in and actions have certain outcomes
 - The agent has multiple possible current states it can't distinguish between
 - Actions have random outcomes
 - There is no initial state
- Why is choosing the right level of abstraction important in problem formulation?
 - Too much detail makes the state space intractably large; too little loses information needed to solve the problem
 - It doesn't matter, any level works
 - Abstraction only affects heuristic design
 - Abstraction is only relevant to adversarial search

18. For a “contingency problem” (nondeterministic or partially observable), what replaces the notion of a simple “solution path”?
- a. A conditional plan / policy that branches based on percepts (b) A single fixed sequence of actions (c) No solution is possible (d) The goal test becomes irrelevant
19. NAT: If a state-space search tree has branching factor 2 and you expand all nodes up to depth 5 (uniform tree, root at depth 0), how many nodes are at depth 5 exactly?
20. Which of these best distinguishes “problem-solving search” from “adversarial search” in terms of state-space formulation?
- a. Adversarial search must also model an opponent’s actions/choices as part of the state transitions, not just the agent’s own actions (b) They are identical (c) Problem-solving search has no goal test (d) Adversarial search never uses a state space

Answer Key — Topic 6

1(a) standard five-component formulation · 2(b) full board configuration · 3(b) average successors per node · 4(a) sum of step costs · 5(a) full combinatorial state description needed · 6(b) transition model definition · 7(a) reachable set definition · 8(c) continuous coordinates → infinite · 9(a) explicit set or property-based · 10(b) $3^4=81$ · 11(a) actual physical actions · 12(a) correct abstraction definition · 13(b) incremental partial-state formulation · 14(a) combinatorial growth · 15(b) real-world cost accumulation · 16(a) definition of single-state problem · 17(a) tradeoff explanation · 18(a) conditional plans for uncertainty · 19. Answer: 32 (2^5) · 20(a) key distinguishing feature

Tip: work each Set A block cold (no notes) exactly like a mini-mock, mark it, then do Set B the same way the next day as your 24-hour revision check for these two days.